Transformer 论文阅读笔记

Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, et al. NIPS 2017

阅读笔记整理

2026 年 2 月 28 日

§1 论文基本信息

发表会议	31st Conference on Neural Information Processing Systems (NIPS 2017)
作者单位	Google Brain, Google Research, University of Toronto
关键词	机器翻译 (Machine Translation)、注意力机制 (Attention Mechanism)、序列转录 (Sequence Transduction)
开源代码	`https://github.com/tensorflow/tensor2tensor`

核心摘要

本文提出了一种全新的简单网络架构——**Transformer**。它完全摒弃了传统的循环神经网络（RNN）和卷积神经网络（CNN），仅依赖于**注意力机制**来计算输入和输出的全局依赖关系。该模型在翻译质量上达到了新的 State-of-the-Art（SOTA），同时具备高度可并行化的特性，将训练时间大幅缩减。

§2 研究背景与动机

2.1 核心问题

在 Transformer 提出之前，主流的序列转录模型主要基于复杂的循环神经网络（RNN）或卷积神经网络（CNN），并采用包含编码器和解码器的架构。

传统架构的困境

- **序列计算的限制：** RNN 模型通常沿着输入和输出序列的符号位置进行因子化计算，生成隐藏状态序列 h_t，这使得训练样本内部无法实现并行化。
- **长距离依赖学习困难：** 在基于卷积的模型（如 Extended Neural GPU, ByteNet, ConvS2S）中，关联两个任意位置信号所需的计算操作数量会随着位置距离的增加而增加，导致学习长距离依赖变得困难。
- **性能与效率的矛盾：** 虽然注意力机制已被广泛应用，但几乎总是与复杂的循环网络结合使用，计算效率依然受限。

2.2 Transformer 的核心思路

用纯粹的自注意力机制替代循环和卷积。在 Transformer 中，关联任意两个位置的操作被减少到了常数级别，并且允许显著提高计算的并行化程度。

§3 方法详解

3.1 模型架构 *(Model Architecture)*

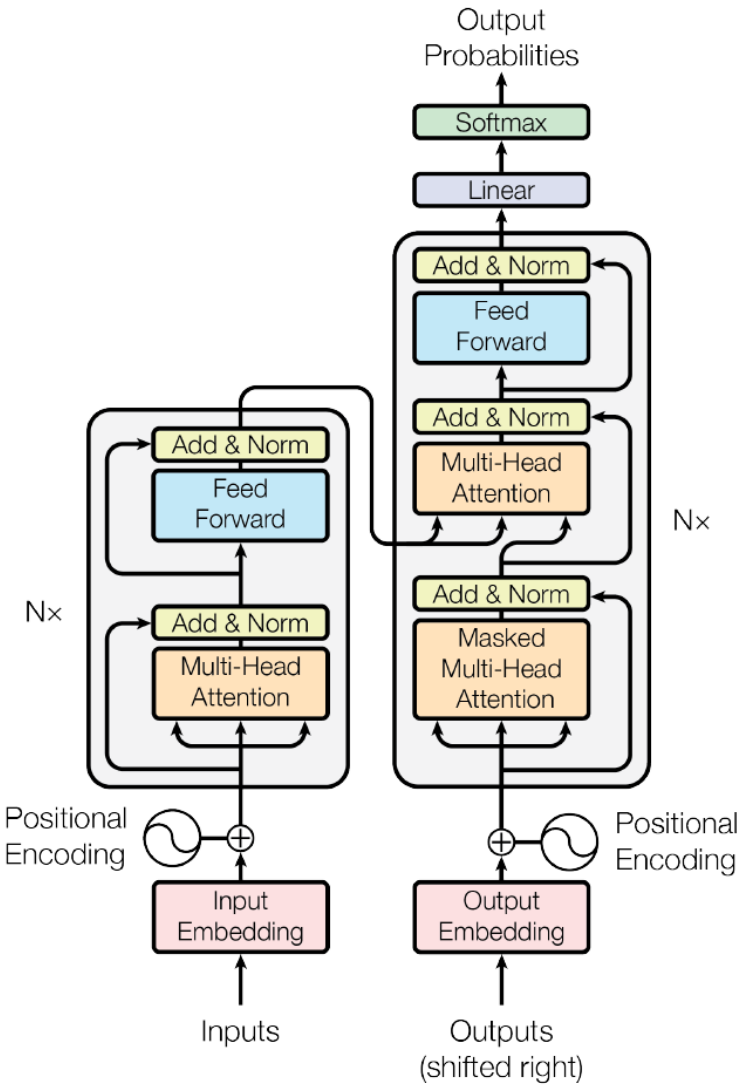

图 1: Transformer 模型架构示意图

Transformer 遵循典型的编码器-解码器结构，但使用堆叠的自注意力和逐位置全连接层。

首先我们可以发现，Transformer 的架构是左右分开的，左侧即为编码器，右侧即为解码器。由于输入和输出长度和语义空间不对齐，所以说会需要一个模块负责输入，另一个模块负责输出。

在编码器中，每个词都可以看到句子中所有的词语（前后文），它的目的是提取上下文信息；而在解码器中，每个词只能够看到它前面的词语，它的目的是生成下一个词语。

编码器与解码器

- **编码器 (Encoder)**：由 $N = 6$ 个相同层堆叠而成。每层包含两个子层：多头自注意力机制（Multi-Head Self-Attention）和位置前馈网络（Feed-Forward Network）。每个子层后都有残差连接和层归一化（Layer Normalization）。所有子层输出维度均为 $d_{model} = 512$。
- **解码器 (Decoder)**：同样由 $N = 6$ 个相同层堆叠而成。除了编码器中的两个子层外，还插入了第三个子层，用于对编码器输出进行多头注意力计算。解码器中的自注意力子层加入了**掩码 (Masking)**，确保位置 i 的预测只能依赖于小于 i 的已知输出，从而保持自回归特性。

模型组件

- **Input/Output Embedding**: 输入和输出词嵌入层共享权重，与编码器和解码器的 d_{model} 维度保持一致，显著减少了参数量。
- **Position Encoding**: 由于模型缺乏循环和卷积结构，必须显式注入序列位置信息。本文采用基于正弦和余弦函数的固定位置编码方案，确保模型能够捕捉序列中的相对位置信息。
- **(Masked) Multi-Head Attention**: 编码器中的每个子层包含一个多头自注意力机制，而解码器中的每个子层则包含两个多头注意力机制：一个用于自注意力，另一个用于编码器-解码器之间的信息交互。
- **Add & Norm**: 每个子层后均添加残差连接和层归一化，既增强了梯度流动，又提升了训练的稳定性。
- **Feed Forward**: 每个子层还包含一个逐位置前馈网络，对每个位置的输出独立应用非线性变换，进一步提升了模型的表达能力。

3.2 注意力机制 *(Attention)*

注意力函数将 Query 和一组 Key-Value 映射到输出，输出是 Value 的加权和。

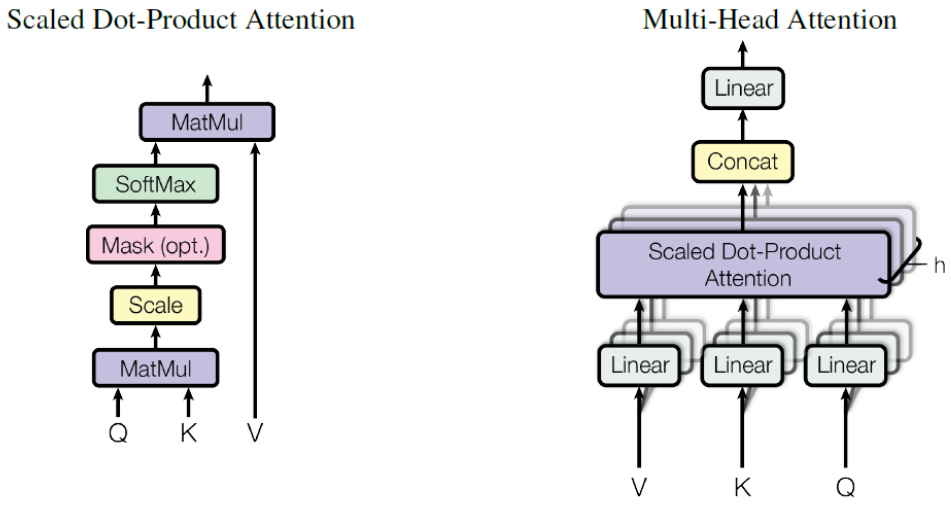

图 2: 注意力机制示意图

3.2.1 缩放点积注意力 *(Scaled Dot-Product Attention)*

输入包含维度为 d_k 的 query 和 key，以及维度为 d_v 的 value。计算矩阵形式如下：

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

注意力机制的核心，就是通过计算 query 和 key 之间的相似度来确定 value 的权重。比如特别相关的那些，就赋予更高的权重。

注意

为什么要除以 $\sqrt{d_k}$？

当 d_k 较大时，点积结果的数量级会变大，推动 softmax 函数进入梯度极小的区域。为了抵消这种影响，通过除以 $\sqrt{d_k}$ 进行缩放。

3.2.2 多头注意力 *(Multi-Head Attention)*

不使用单一的注意力函数，而是将 queries, keys, values 分别进行 h 次不同的线性投影，投影到 d_k, d_k 和 d_v 维度。

- 在投影后的表示上并行执行注意力函数，产生 d_v 维输出。
- 允许模型在不同位置联合关注来自不同表示子空间的信息。
- 本文使用 $h = 8$ 个平行的注意力头，由于每个头降维（$d_k = d_v = d_{model}/h = 64$），总计算成本与全维度的单头注意力相似。

使用多头注意力的原因：通过并行的多个独立的注意力机制，模型就可以从不同的表示子空间去学习相关的信息。

3.3 前馈网络与位置编码 *(FFN & Positional Encoding)*

3.3.1 逐位置前馈网络 *(Position-wise FFN)*

每个编码器和解码器层都包含一个完全连接的前馈网络，对每个位置单独且等同地应用。包含两次线性变换与 ReLU 激活：

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

输入输出维度为 $d_{model} = 512$，中间层维度为 $d_{ff} = 2048$。

引入中间层维度 2048 的原因，是希望通过增加非线性变换的容量来提升模型的表达能力。虽然输入输出维度保持不变，但中间层的扩展允许模型捕捉更复杂的特征和关系，从而增强整体性能。

3.3.2 位置编码 *(Positional Encoding)*

由于模型缺少循环和卷积，必须显式注入序列相对或绝对位置的信息。本文采用不同频率的正弦和余弦函数：

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}}) \tag{3}$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}}) \tag{4}$$

选择该函数是假设它能让模型容易地学习到相对位置的注意力机制，因为对于任意固定偏移量 k，PE_{pos+k} 均可表示为 PE_{pos} 的线性函数。

这样在低频的时候，位置编码的变化较慢，适合捕捉长距离依赖；而在高频的时候，位置编码的变化较快，适合捕捉短距离依赖。这种设计使得模型能够同时关注不同范围的上下文信息。

§4 与循环层、卷积层的深度对比

Transformer vs. RNN / CNN

层类型	每层计算复杂度	顺序执行操作数	最大路径长度
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent (RNN)	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$

- **并行度**：自注意力机制连接所有位置只需要常数级的连续执行操作数（$O(1)$），而 RNN 需要 $O(n)$。
- **长程依赖**：学习长距离依赖的一个关键因素是网络中前后向信号必须穿过的路径长度。自注意力层的最大路径长度为 $O(1)$，相比 RNN / CNN 极大简化了长程依赖的学习。

§5 实验与结果

5.1 机器翻译表现

模型	BLEU (EN-DE)	BLEU (EN-FR)
GNMT + RL Ensemble	26.30	41.16
ConvS2S Ensemble	26.36	41.29
Transformer (base)	**27.3**	**38.1**
Transformer (big)	**28.4**	**41.8**

- **翻译质量打破记录**：在 WMT 2014 英德翻译任务上，Transformer (Big) 达到了 28.4 BLEU，超出此前最佳模型（包括集成模型）2.0 以上。在英法翻译上达到了 41.8 的最新单模型 SOTA。
- **极高的训练效率**：大型模型在 8 张 P100 GPU 上仅训练了 3.5 天，训练成本仅为以往竞争模型的一小部分。

5.2 泛化能力 (English Constituency Parsing)

即便没有针对任务做特定微调，使用 4 层 Transformer 依然在 Penn Treebank 的 WSJ 数据集上泛化良好，半监督设置下 F1 分数达到 92.7，优于传统 RNN 序列转录模型。

§6 消融实验探讨

为了验证各个组件的重要性，作者在英德翻译验证集上进行了大量变量控制实验。

- **注意力头数 (h)**：单头注意力（h=1）性能比最佳设置下降 0.9 BLEU，但头部过多同样会导致性能下降。
- **Key 的维度 (d_k)**：降低 d_k 尺寸会伤害模型质量，这表明兼容性计算并非易事，过于简单的点积可能不够用。
- **模型规模与正则化**：更大的模型能带来更好的表现；Dropout 对于防止过拟合极其有效。
- **位置编码替换**：将正弦位置编码替换为可学习的位置嵌入，得到了与基础模型几乎一致的结果。

§7 总结

核心技术贡献

1. **纯注意力网络：**提出了第一个完全基于注意力机制的序列转录模型，摆脱了以往机器翻译领域对 RNN 和 CNN 的依赖。
2. **训练速度跃升：**消除了计算的序列依赖性，使架构能够显著加快训练速度并超越之前的所有集成模型。
3. **注意力机制的可解释性：**附加收益在于自注意力带来了更高的模型解释度。不同的注意力头学习执行不同的任务，并呈现出与句子句法及语义结构相关的行为。

§8 个人理解与思考

Transformer 的创新与影响

- **架构创新：**Transformer 的核心创新在于完全摒弃了传统的循环和卷积结构，转而使用多头自注意力机制。这种设计不仅简化了模型结构，还极大提升了并行计算能力。
- **性能提升：**在多个机器翻译任务上，Transformer 显著超越了之前的 SOTA 模型，证明了其在处理长距离依赖和复杂语言结构方面的优势。
- **广泛应用：**Transformer 的成功引发了 NLP 领域的革命，成为后续众多模型（如 BERT、GPT）的基础架构，并被广泛应用于文本生成、理解、翻译等任务。

Transformer 如何解决序列建模中“并行化”和“长距离依赖”的痛点？

- **RNN**:RNN 是序列模型的主流，本质是顺序计算。当前时间步的隐藏状态 h_t 依赖于前一时间步的隐藏状态 h_{t-1}，这导致了计算的串行化，无法并行处理。而 Transformer 通过自注意力机制，允许所有位置同时计算注意力权重，从而实现了并行化。
- **CNN**:CNN 虽然可以并行，但是其感受野受限于卷积核的大小。如果要关联距离比较远的两个位置，那么就需要堆叠多层或者使用空洞卷积。这就会导致最大路径的长度呈现对数或者线性增长，信息传递的路径越长，学

习就会越来越难。

- **Transformer 的解决方案**：Transformer 通过采用自注意力机制，一次性接收整个序列，通过矩阵运算，同时计算序列中所有位置之间的注意力权重。这就使得序列操作数为常数级 O(1)。同时在自注意力机制当中，任意两个位置都可以通过一次注意力计算直接交互，实现计算的高效并行，并且实现了信息的直接传递。

位置编码探究

- **为什么需要位置编码？**：注意力机制本身是置换不变的，也就是说如果将输入序列的词序打乱，对于自注意力机制来说，计算出的注意力矩阵虽然会因为位置的不同而不同，但是模型的本质是在处理一个“词袋集合”，而非“词序列”，也就是说它不知道“猫追老鼠”和“老鼠追猫”的区别。因为词和词的绝对和相对位置信息完全丢失了。因此，必须显式地将位置信息注入模型

- **正弦/余弦位置编码的物理意义**：这种设计的第一个优点是对于任意的固定偏移量 k,PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数，这使得模型能够轻松地学习到相对位置的注意力机制。第二个优点是不同频率的正弦和余弦函数能够让模型同时关注不同范围的上下文信息，低频部分适合捕捉长距离依赖，高频部分适合捕捉短距离依赖。这种设计使得模型能够更好地理解序列中的结构和关系。

如何从宏观上来理解 Transformer 架构中的三大注意力子层？

- **编码器自注意力**: 这个子层的作用是让编码器在处理某一个词的时候，能够看到整个输入的句子。

- **解码器掩码自注意力**: 这个子层的作用是让解码器在生成某一个词的时候，只能看到它前面已经生成的词，不能看到后面的词。这样就保证了模型的自回归特性。

- **编码器-解码器注意力**: 这个子层连接编码器和解码器。解码器中的每一个位置都会向编码器输出序列进行查询。这就让解码器在生成每一个词的时候，都可以关注输入序列中所有相关的词。